Stealthy LLM-Driven Data Poisoning Attacks Against Embedding-Based Retrieval-Augmented Recommender Systems

Fatemeh Nazary*
Polytechnic University of Bari
Department of Electrical and Information Engineering (DEI)
Bari, Italy
fatemeh.nazary@poliba.it

Yashar Deldjoo
Polytechnic University of Bari
Department of Electrical and Information Engineering (DEI)
Bari, Italy
yashar.deldjoo@poliba.it

Tommaso Di Noia
Polytechnic University of Bari
Department of Electrical and Information Engineering
Bari, Italy
tommaso.dinoia@poliba.it

Eugenio Di Sciascio
Polytechnic University of Bari
Department of Electrical and Information Engineering
Bari, Italy
eugenio.disciascio@poliba.it

Abstract

We present a systematic study of *provider-side* data poisoning in retrieval-augmented recommender systems (RAG-based). By modifying only a small fraction of tokens within item descriptions—for instance, adding emotional keywords or borrowing phrases from semantically related items—an attacker can significantly promote or demote targeted items. We formalize these attacks under token-edit and semantic-similarity constraints, and we examine their effectiveness in both *promotion* (long-tail items) and *demotion* (short-head items) scenarios. Our experiments on MovieLens, using two large language model (LLM) retrieval modules, show that even subtle attacks shift final rankings and item exposures while eluding naive detection. The results underscore the vulnerability of RAG-based pipelines to small-scale metadata rewrites, and emphasize the need for robust textual consistency checks and provenance tracking to thwart stealthy provider-side poisoning.

Keywords

Retrieval-Augmented Generation, Recommender Systems, Data Poisoning, Large Language Models, Adversarial Text Attacks

ACM Reference Format:
Fatemeh Nazary, Yashar Deldjoo, Tommaso Di Noia, and Eugenio Di Sciascio. 2025. Stealthy LLM-Driven Data Poisoning Attacks Against Embedding-Based Retrieval-Augmented Recommender Systems. In *Adjunct Proceedings of the 33rd ACM Conference on User Modeling, Adaptation and Personalization (UMAP Adjunct '25), June 16–19, 2025, New York City, NY, USA.* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3708319.3733675

*Corresponding author.

UMAP Adjunct '25, New York City, NY, USA

ACM ISBN 979-8-4007-1399-6/25/06
https://doi.org/10.1145/3708319.3733675

1 Introduction

Retrieval-augmented generation (RAG) enhances large language models (LLMs) by grounding their outputs in external data sources, such as item reviews or user tags, rather than relying solely on internal parameters [2, 3, 6]. This grounding improves recency and factual accuracy. In fact, industry reports estimate that over 60% of LLM-powered search and recommendation systems now use RAG [6, 8]. A common RAG-based recommender architecture (Figure 1) retrieves candidate items from an external knowledge store (e.g., a database or corpus of item descriptions) and then uses an LLM to synthesize the retrieved text into final recommendations. While classical methods such as collaborative filtering (CF) can provide a base in the retrieval stage by analyzing user-item interactions, **embedding-based retrieval** has emerged as a particularly powerful approach in RAG pipelines. Embedding models such as BERT-like encoders or Sentence Transformers can capture an item semantic representation and dynamically decide when to retrieve additional context. By leveraging these embeddings, a recommender system can handle *new* or *infrequently* discussed items, which often appear in long-tail domains. At the same time, these embedding-based retrieval methods offer clear advantages: they provide stronger factual grounding and can adapt to real-time changes in external data. For example, if an item has recently won an award, the RAG pipeline can incorporate that information into the recommendations without retraining the entire model. Notwithstanding their great potential, as more RAG variants adopt embedding-driven approaches, they lean heavily on textual cues, which can open the door to attacks at the data or metadata level. For example, an attacker can subtly *inject* changes into item descriptions (e.g. emotional phrases, negative triggers) to manipulate how both retrieval and generation perceive an item. Unlike classic poisoning that directly tampers with user ratings, *text-based* attacks may remain undetected if they preserve the original semantics.

Related Work and Gaps. LLM vulnerabilities to prompt injection and adversarial prompts have been well documented [1, 7, 10, 11], however, these studies focus on standalone models rather than

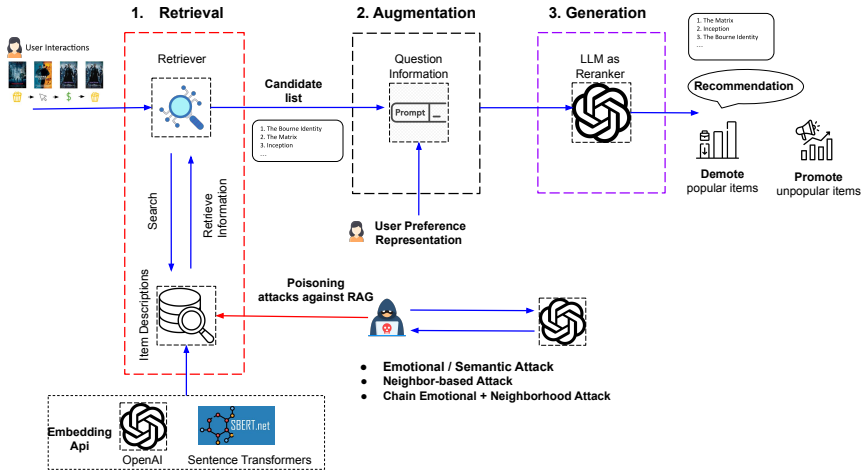

Figure 1: High-level RAG architecture in a recommender setting. A retriever selects candidate items (step 1). An LLM uses these retrieved texts and user queries to re-rank or generate final recommendations (step 2). In our *poisoning* scenario (red arrow), an attacker subtly modifies item descriptions to alter how retrieval and generation perceive items.

recommendation pipelines. In the recommender domain, early poisoning attacks manipulated user ratings or profiles to distort collaborative filtering outputs [5]. More recently, RAG–specific threats emerged: BadRAG [12] and PoisonedRAG [13] inject malicious snippets into knowledge bases to warp retrieval or LLM responses. Tag-based poisoning for RAG recommenders was explored in [8], showing that modified user tags can bias item ranking. However, full metadata, such as item descriptions and reviews, remains underexamined. Furthermore, prior methods often rely on simple keyword insertion, which can be detected by basic semantic or stylistic filters. In contrast, we leverage modern LLM rewriting techniques to design coherent, low-budget alterations that better evade detection [4].

Goals and Contributions Our primary objective is to formally investigate *provider-side* data poisoning attacks on retrieval-augmented recommenders, focusing on two scenarios: *promotion* (boosting long-tail items) and *demotion* (penalizing highly popular items). In doing so, we introduce the notion of "textual stealthiness," measured through semantic similarity (e.g., SBERT-based) along with overall system-level metrics, to quantify how much an attack can subtly rewrite an item's description while still achieving malicious ranking shifts. Our main contributions include:

- We present a formal definition of provider-side textual rewriting attacks in RAG-based recommendation, framed around two adversarial goals (*promote* vs. *demote*).
- We use a measurement of "stealthiness" by examining how sentence-level semantics change (via SBERT) alongside the overall impact on RS accuracy;
- We design and implement *three* distinct attack variations: (i) *Emotional* edits, (ii) *Neighbor-based* borrowing, and (iii) *Chained* rewriting (combining both), all under the same edit-budget constraints.
- We empirically evaluate these methods across *two different SoA LLMs* on the of the MovieLens latest dataset, and demonstrate the potency of small-scale textual manipulations on altering the exposure of carefully selected target items;
- We plan to release both *code* and *data* resources to foster reproducibility and future research in adversarial robustness for RAG-based recommender systems.

2 Formal Description of LLM-Driven Data Poisoning Attacks

The overarching goal of the attacker is to manipulate item visibility within the recommendation pipeline by rewriting the textual descriptions of items. Concretely, we select a subset of items from both the long-tail (unpopular) and short-head (popular) segments, aiming to *promote* the former (i.e., increase their exposure) or *demote* the latter (i.e., decrease their visibility). Formally, for each targeted item $i \in I_{\text{poison}}$ with original description D_i, we produce a new description $\widetilde{D}_i$ such that (1) the token-level change is bounded by δ (e.g., 10% of $|D_i|$ tokens may be altered), and (2) the rewritten text maintains a sufficiently high semantic similarity (e.g., SBERT score above 0.80) to remain stealthy. The attacker's optimization objective is then to maximize (in the promote case) or minimize (in the demote case) each item's final ranking position or exposure in top-k recommendations after the system retrains on $\widetilde{D}_i$:

$$\begin{aligned} \max_{\{\widetilde{D}_i\}} \sum_{i \in I_{\text{poison}}} \Delta\Big(\text{Exposure}(i)\Big) \\ \text{subject to} \quad H(D_i, \widetilde{D}_i) \le \delta |D_i|, \\ \text{Sim}(D_i, \widetilde{D}_i) \ge \sigma_{\min}. \end{aligned} \tag{1}$$

Here, $\Delta(.)$ denotes the change in ranking of item i within the ranking list (either at the retrieval level with top-N or the final recommendation level with top-k). In our experiment, we set $N = 50$ and $K = 20$. The function $H(\cdot)$ represents a distance metric, where δ

introduces the notion of **stealthiness**, ensuring that modifications remain subtle yet effective. We specifically instruct the LLM to modify 10% of the tokens (token-level distance) while measuring the change at both the *token level*, and the *semantic level* using Sentence-BERT (SBERT)[9]. In our framework, we designed 3 types of attacks:

- **Emotional Attack:** We prompt the LLM to analyze what textual cues make a movie appear "more popular" or "less popular," given its original description. The LLM then modifies up to 10% of the text, injecting emotive or sentiment-laden words (e.g., "exhilarating," "lackluster") to steer the item toward promotion or demotion.
- **Neighbor Borrowing:** We identify a set of *n neighbor* items from the opposite popularity segment (i.e., from popular to help promote, or from unpopular to help demote). The LLM then weaves phrases from these neighbors into the target's description, aiming to transfer either highly positive or negative connotations. In our experiments, $n = 5$.
- **Chain Attack:** This approach combines both emotional triggers and neighbor-based snippets in a single pass. While often more powerful, the resulting text can become more visibly altered relative to the original description.

Example of Metadata Attack:

Original Description:

> *"Wings of Hope lifts spirits in trying times. Courage takes flight."*

Emotional Attack (Promote):

> "Wings of Hope **soars** to lift spirits in trying times, where courage takes **exhilarating** flight. *Uplifting Impact: Embrace the strength within to rise above challenges.*"

Neighbor Borrowing (Promote):

> "Wings of Hope lifts spirits in trying times. *Experience how Cyber Love blends technology with passion, creating a harmonious blend of optimism and connection.* Courage takes flight."

Chain Attack:

> Merges emotive language (e.g., "exhilarating," "uplifting") *and* references to similar "popular" neighbors to maximize the embedding shift, e.g.: "Wings of Hope **exhilaratingly** showcases resilience *(inspired by Cyber Love's uplifting themes)* to soar beyond challenges."

3 Experimental Setup

We conduct our experiments on top of a retrieval-augmented recommendation (RAG) pipeline that integrates a Large Language Model (LLM) from OpenAI[1] and a Sentence Transformer (ST)[2] encoder for embedding-based retrieval. Specifically, the pipeline first employs the Sentence Transformer to select candidate items based on the semantic closeness of item descriptions, and then an OpenAI-based LLM re-ranks these candidates or generates final textual recommendations.

[1]Using the official API for text generation and re-ranking
[2]https://www.sbert.net/

Although we report results primarily on the widely used MovieLens "ml-latest-small" dataset in this paper, we have also evaluated the proposed methods on additional benchmark datasets; due to space limitations, we present here only the detailed findings for the MovieLens dataset.

Within the MovieLens corpus, we categorize items into *long-tail* (unpopular) vs. *short-head* (popular) segments, inject adversarial textual edits, and assess the resulting changes in item ranks and system-level metrics (e.g., Recall@k, nDCG@k). Our system reindexes or retrains on these modified descriptions, thereby simulating real-world scenarios where metadata updates could inadvertently (or maliciously) be incorporated into a live recommender. We build the user profile for the retrieval stage in two ways: (1) **Manual** construction using a structured template; and (2) **LLM-based** summarization that generates a user's preferences automatically. In Table 1 **(Tab 2 and 3),** we differentiate these two methods when evaluating final recommendations. All remaining steps in our pipeline (retrieval and re-ranking) remain unchanged.

4 Results and Discussion

We now present our key experimental findings, structured around these three main research questions (RQs).

RQ1: Are LLM-based textual attacks effective at pushing a target item ranking up or down (both at retrieval top-N and recommendation stage top-k)?
RQ2: Does attack efficacy vary across LLMs model e.g., OpenAI vs. Sentence Transformer retrieval?
RQ3: How do these modifications affect overall recall or nDCG? Can poisoning degrade system-wide performance?

RQ1: Effectiveness of LLM-Based Textual Attacks

Table 1 (top half) presents results for the *promotion* scenario. The bold "Original" rows provide baseline ranks against which each attack variant (Emotional, Neighborhood, Chain) can be compared. A successful promotion reduces the rank value, indicating an item is placed closer to the top of recommended lists. In several cases, Chain rewriting reduces ranks from approximately 7.0 to around 4.7, whereas Emotional and Neighbor approaches achieve more moderate improvements. These findings validate that even a modest injection of sentiment-laden descriptors or borrowed phrases significantly influences item visibility.

Turning to the *demotion* scenario (Table 1 bottom half), the goal is to push popular items into lower positions (hence, a successful attack increases rank). Chain-based edits again elicit the largest rank changes, demonstrating the capacity of compound strategies—merging emotional cues with neighbor-based snippets—to degrade targeted items more substantially. Thus, we conclude that small-scale textual rewrites are demonstrably effective in shifting final recommendations.

Table 1: Side-by-side comparison of Promotion (top) and Demotion (bottom) scenarios for a temporal pipeline. Each scenario lists (A) Retrieval (temporal), (B) Recommendation (LLM-based profile), and (C) Recommendation (Manual profile). Columns show OpenAI vs. Sentence Transformer (ST), with Ranking of attacked items (lower = stronger promotion). Bold is the base (no attack), cyan highlights best results, yellow highlights good results.

Scenario / Pipeline		OpenAI			ST		
		Rank	Recall	nDCG	Rank	Recall	nDCG
Promotion Scenario							
(A) Retrieval	Original	**31.53**	0.1504	0.2101	**21.01**	0.1205	0.1615
	Emotional	28.65	0.1289	0.1920	33.00	0.1191	0.1752
	Neighborhood	28.54	0.1486	0.2047	29.23	0.1196	0.1698
	Chain	**25.16**	0.1367	0.1968	32.16	0.1241	0.1669
(B) Rec. (LLM)	Original	**7.00**	0.0944	0.1808	**5.27**	0.0701	0.1561
	Emotional	8.25	0.0793	0.1838	8.00	0.0758	0.1634
	Neighborhood	6.67	0.0739	0.1546	6.40	0.0652	0.1573
	Chain	**4.67**	0.0830	0.1749	7.50	0.0699	0.1548
(C) Rec. (Manual)	Original	**6.50**	0.0853	0.1823	**4.92**	0.0761	0.1583
	Emotional	6.00	0.0834	0.1847	8.50	0.0749	0.1616
	Neighborhood	**3.00**	0.0804	0.1695	7.33	0.0661	0.1417
	Chain	5.89	0.0834	0.1743	**1.00**	0.0725	0.1684
Demotion Scenario							
(A) Retrieval	Original	**25.56**	0.1504	0.2101	**26.99**	0.1205	0.1615
	Emotional	22.80	0.1299	0.1849	21.69	0.1246	0.1679
	Neighborhood	24.66	0.1414	0.1929	25.13	0.1190	0.1674
	Chain	20.60	0.1344	0.1931	25.91	0.1251	0.1637
(B) Rec. (LLM)	Original	**5.72**	0.0943	0.1842	**4.95**	0.0733	0.1543
	Emotional	4.95	0.0853	0.1966	4.40	0.0755	0.1700
	Neighborhood	**5.87**	0.0790	0.1759	4.75	0.0793	0.1794
	Chain	5.44	0.0747	0.1760	4.44	0.0707	0.1599
(C) Rec. (Manual)	Original	**5.65**	0.0972	0.1837	**4.96**	0.0767	0.1694
	Emotional	5.53	0.0810	0.1857	4.57	0.0670	0.1514
	Neighborhood	5.63	0.0765	0.1765	4.67	0.0817	0.1703
	Chain	5.38	0.0740	0.1748	**5.00**	0.0716	0.1488

RQ2: Comparison of OpenAI vs. Sentence Transformer Retrieval

An additional observation arises when contrasting the OpenAI columns with the Sentence Transformer (ST) columns. The OpenAI-based pipeline exhibits increased sensitivity to the introduced textual modifications. For instance, in the *promotion* scenario, a change from a rank of 7.0 to approximately 4.7 is relatively large, whereas the corresponding ST scenario occasionally reverses the direction of movement or produces more modest variation. These disparities highlight how generative re-ranking can amplify subtle language cues or signals introduced by adversarial text rewriting. Moreover, the propensity of OpenAI to rely on nuanced phrasing suggests that even brief "trigger" terms can be disproportionately influential, especially compared to a more static embedding architecture.

RQ3: Impact on System-Wide Recall and nDCG

Beyond item-specific ranking, Table 1 also reports Recall and nDCG. Notably, these global performance metrics do not consistently suffer drastic declines, with the maximum observed drop typically limited to a few percentage points. While such localized attacks primarily disrupt the visibility of a targeted subset, large-scale poisoning—where a significant fraction of items are manipulated—could lead to more pervasive performance deterioration. This aligns with related work demonstrating that simultaneous metadata rewrites on a broader scale can substantially undermine system accuracy [12]. In short, although the system's global fidelity remains relatively intact for sparse attacks, the localized impact on individual item positions seems to be more pronounced.

Overall, these results confirm that retrieval-augmented recommender systems are vulnerable to data poisoning via concise textual edits. Small-scale, stealthy manipulations—particularly those that combine emotive triggers and neighbor-based phrasing—can effectuate substantial ranking shifts without severely compromising system-level metrics. The heightened sensitivity of LLM-driven pipelines underscores the importance of developing robust checks on textual provenance and integrity to mitigate provider-side poisoning attempts.

5 Conclusion

Our work demonstrates that carefully designed textual perturbations (modifications) in item metadata can strategically alter recommendations in Retrieval-Augmented Generation (RAG) systems, emphasizing the need for robust textual provenance checks. Through a systematic exploration of different attack strategies—including *emotional rewording*, *neighbor-based borrowing*, and *hybrid chaining*—our experiments reveal that even **small-scale semantic manipulations** can effectively boost the visibility of long-tail items or suppress popular ones, often while remaining stealthy and difficult to detect. These findings underscore the potential provider-side vulnerabilities in RAG-based pipelines and the necessity of defensive measures to safeguard recommendation integrity.

Acknowledgments

The authors acknowledge partial support of the following projects: OVS: Fashion Retail Reloaded and Lutech Digitale 4.0.

References

[1] Arijit Ghosh Chowdhury, Md Mofijul Islam, Vaibhav Kumar, Faysal Hossain Shezan, Vinija Jain, and Aman Chadha. 2024. Breaking down the defenses: A comparative survey of attacks on large language models. *arXiv preprint arXiv:2403.04786* (2024).

[2] Yashar Deldjoo, Zhankui He, Julian McAuley, Anton Korikov, Scott Sanner, Arnau Ramisa, René Vidal, Maheswaran Sathiamoorthy, Atoosa Kasirzadeh, and Silvia Milano. 2024. A Review of Modern Recommender Systems using Generative Models (Gen-RecSys). In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 6448–6458.

[3] Yashar Deldjoo, Zhankui He, Julian McAuley, Anton Korikov, Scott Sanner, Arnau Ramisa, Rene Vidal, Maheswaran Sathiamoorthy, Atoosa Kasrizadeh, Silvia Milano, et al. 2024. Recommendation with Generative Models. *arXiv preprint arXiv:2409.15173* (2024).

[4] Yashar Deldjoo, Nikhil Mehta, Maheswaran Sathiamoorthy, Shuai Zhang, Pablo Castells, and Julian McAuley. 2025. Toward Holistic Evaluation of Recommender Systems Powered by Generative Models. *SIGIR'25* (2025).

[5] Yashar Deldjoo, Tommaso Di Noia, and Felice Antonio Merra. 2021. A survey on adversarial recommender systems: from attack/defense strategies to generative adversarial networks. *ACM Computing Surveys (CSUR)* 54, 2 (2021), 1–38.

[6] Wenqi Fan, Yujuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, and Qing Li. 2024. A survey on rag meeting llms: Towards retrieval-augmented large language models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 6491–6501.

[7] Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Zihao Wang, Xiaofeng Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, et al. 2023. Prompt Injection attack against LLM-integrated Applications. *arXiv preprint arXiv:2306.05499* (2023).

[8] Fatemeh Nazary, Yashar Deldjoo, and Tommaso di Noia. 2025. Poison-rag: Adversarial data poisoning attacks on retrieval-augmented generation in recommender systems. In *European Conference on Information Retrieval*. Springer, 239–251.

[9] N Reimers. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *arXiv preprint arXiv:1908.10084* (2019).

[10] Yifei Wang, Dizhan Xue, Shengjie Zhang, and Shengsheng Qian. 2024. BadAgent: Inserting and Activating Backdoor Attacks in LLM Agents. *arXiv preprint arXiv:2406.03007* (2024).

[11] Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. 2024. Jailbroken: How does llm safety training fail? *Advances in Neural Information Processing Systems* 36 (2024).

[12] Jiaqi Xue, Mengxin Zheng, Yebowen Hu, Fei Liu, Xun Chen, and Qian Lou. 2024. BadRAG: Identifying Vulnerabilities in Retrieval Augmented Generation of Large Language Models. *arXiv preprint arXiv:2406.00083* (2024).

[13] Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. 2024. Poisonedrag: Knowledge poisoning attacks to retrieval-augmented generation of large language models. *arXiv preprint arXiv:2402.07867* (2024).